



Dompé

DOCUMENTAZIONE TECNICA

Low Level Design

Struttura del codice, moduli, contratti interni, configurazione e strategia di test



Campo	Valore
Applicativo	Guest Portal — Cisco Catalyst 9800 Guest Management Desk
Indirizzo (produzione)	https://guestportal.dompe.com
Documento	LLD — Low Level Design
Versione	1.2
Data di emissione	25 settembre 2026
Stato	Emesso
Destinatari	IT Infrastructure, Security, Sviluppo, Service Desk
Classificazione	Uso interno
Repository di riferimento	github.com/ACF-Dompe/Centralino
Revisione del codice	main · 3338398 (+ modifiche non ancora committate)

Indice

1	Scopo e convenzioni	4
1.1	Convenzioni	4
1.2	Struttura del repository	4
2	Vista dei moduli	5
3	Backend.....	6
3.1	Avvio e ciclo di vita del processo.....	6
3.1.1	Arresto controllato.....	6
3.2	Configurazione.....	6
3.2.1	Risoluzione della password del controller	7
3.3	Accesso al database	8
3.4	Autenticazione SAML	8
3.4.1	Cache degli identificativi AuthnRequest	9
3.4.2	Estrazione delle informazioni di identità	9
3.5	Sessione	9
3.6	Accesso di emergenza	9
3.7	Autorizzazione	10
3.7.1	Il profilo di autorizzazione	11
3.7.2	Cache e propagazione delle modifiche	11
3.7.3	Guardie	11
3.7.4	Amministratori per convenzione	12
3.8	Router e gestori	12
3.8.1	Persistenza della sessione prima della risposta.....	12
3.9	Dialogo con i controller	12
3.9.1	Verifica delle credenziali via HTTPS	12
3.9.2	Esecuzione dei comandi via SSH.....	13
3.9.3	Sanitizzazione	13
3.10	Generazione delle credenziali	14
3.11	Invio delle credenziali	14
3.12	Ricerca nella directory Entra ID.....	15
3.13	Password dei controller in Key Vault.....	15
3.14	Servizi di background	16
3.15	Provisioning degli account ospite	17
3.16	Presenza dei dispositivi	17
3.17	Canale WebSocket.....	18

3.18 Registrazione degli eventi	18
3.19 Strumenti da riga di comando	18
4 Frontend	20
4.1 Avvio e macchina a stati	20
4.2 Client API	20
4.3 Componenti	21
4.4 Aggiornamento in tempo reale	21
4.5 Internazionalizzazione	21
4.6 Runtime statico	21
5 Contratti dati	22
6 Errori e codici applicativi.....	23
7 Test.....	24
8 Immagini	25
9 Osservabilità.....	26

1 Scopo e convenzioni

Questo documento descrive come è fatto l'applicativo: quali moduli esistono, che cosa fa ciascuno, quali contratti si scambiano e perché alcune scelte implementative sono quelle che sono. Presuppone la lettura del documento HLD.

1.1 Convenzioni

- I percorsi sono relativi alla radice del repository; `backend/src/...` e `frontend/src/...`
- Il codice è **TypeScript** con moduli **ESM** su entrambi i lati; gli import interni del backend portano l'estensione `.js` perché il codice compilato è ESM nativo.
- I nomi di dominio in codice e in banca dati sono in italiano dove riflettono il dominio aziendale (`sedi`, `sede_id`); il resto è in inglese.
- «Sede» indica una località con il proprio controller; è l'unità di isolamento dei dati.

1.2 Struttura del repository

·	
— backend/	workspace npm – API Express
— src/	
— auth/	strategia SAML, sessione, password, modello di autorizzazione
— db/	pool, migrazioni, dati di riferimento, bootstrap break-glass
— middleware/	ensureAuth, authorize
— repositories/	mappatura riga ↔ dominio
— routes/	index (API operativa), auth, admin
— scripts/	CLI breakglass e appusers
— services/	wlcSsh, wlcWebui, graphMail, email, timer, ws, entraDirectory, wlcCredentials, wlcGuest, wlcPresence
— utils/	credenziali, sanitizzazione, tempo, allowlist IP, throttle, azureCredential, wlcPasswordCache
— config.ts	configurazione centralizzata da variabili d'ambiente
— logger.ts	pino + Application Insights opzionale
— index.ts	avvio, middleware, montaggio router, shutdown
— frontend/	workspace npm – SPA React
— src/	componenti, client API, WebSocket, i18n
— e2e/	Playwright (calendar, rbac, sso)
— nginx/default.conf	configurazione del runtime statico
— .github/workflows/	CI, deploy, sicurezza, e2e, preflight infrastruttura
— Dockerfile	immagine backend (multi-stage)
— Dockerfile.frontend	immagine frontend (nginx unprivileged)
— Makefile	scorciatoie di sviluppo e CLI operative

La radice è un **workspace npm** con due pacchetti (`backend`, `frontend`) e un unico lockfile: le due immagini si costruiscono dallo stesso albero di dipendenze risolto.

2 Vista dei moduli

• Mappa dei componenti applicativi

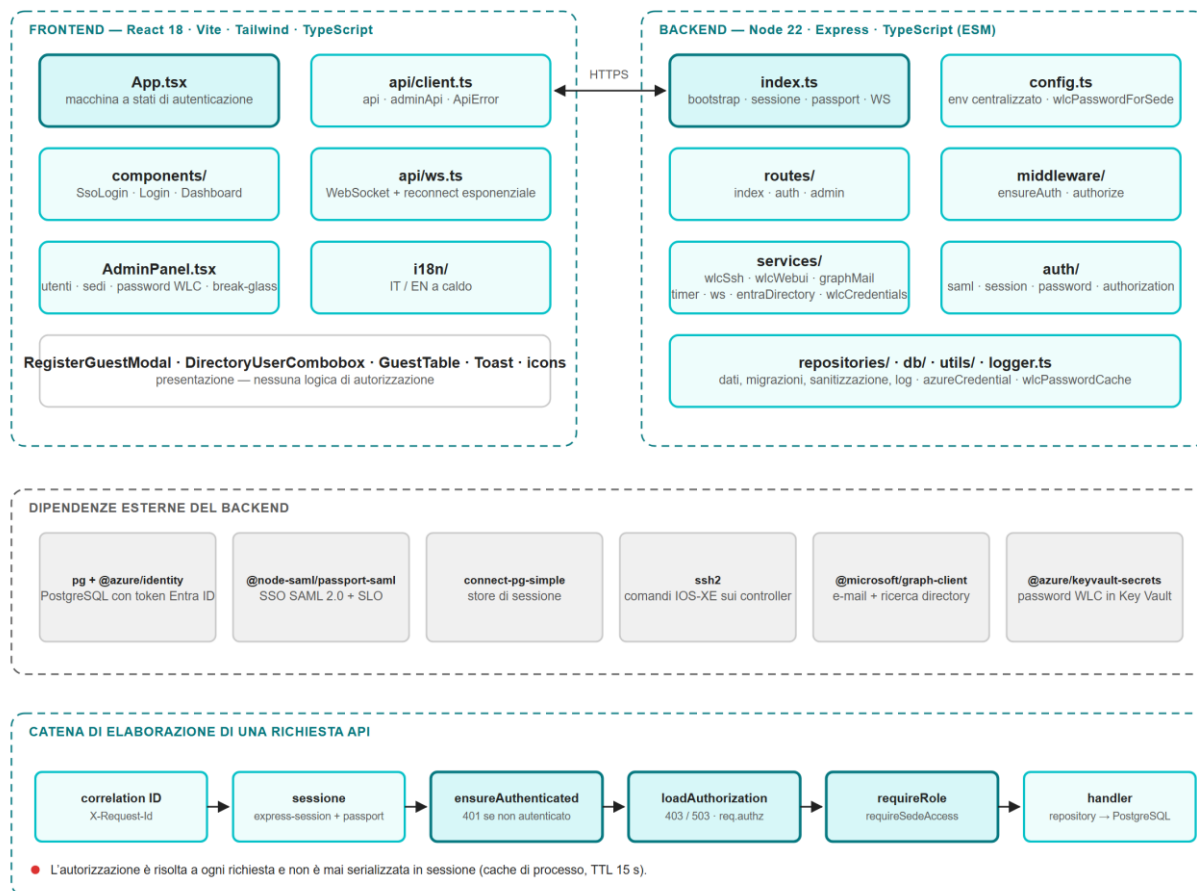


Figura 1 — Moduli del frontend e del backend, dipendenze esterne e catena di elaborazione di una richiesta API.

3 Backend

3.1 Avvio e ciclo di vita del processo

`backend/src/index.ts` è l'unico punto in cui i pezzi vengono messi insieme. L'ordine non è casuale.

1. Apertura del pool verso PostgreSQL e, se abilitate, migrazioni e dati di riferimento.
2. `initWlcPasswords()`: se `KEY_VAULT_URL` è impostato, legge dal vault la password di ogni sede e la porta in memoria. **Non solleva mai eccezioni** — un'indisponibilità di Key Vault non deve impedire l'avvio, e le variabili d'ambiente restano come ripiego.
3. `trust proxy` a 1: l'applicazione sta dietro l'Application Gateway e deve leggere l'indirizzo di origine reale — da cui dipendono la lista IP consentiti e la limitazione per indirizzo.
4. Compressione e parser JSON con limite di 1 MB.
5. Assegnazione di un **identificativo di correlazione** a ogni richiesta, restituito anche nell'intestazione `X-Request-Id`.
6. Creazione dello store di sessione — **uno solo**, condiviso fra il middleware Express e il verificatore dell'upgrade WebSocket.
7. Inizializzazione di passport e della strategia SAML (che può restituire `null` se il certificato dell'IdP è inutilizzabile).
8. Registrazione del log delle richieste **prima** di ogni router, così che anche `/api/auth/*` compaia nei log.
9. Montaggio dei router: `/api/auth`, poi `/api/admin` dietro i propri controlli, infine `/api`.
10. Gestore d'errore JSON in coda, riconoscibile dalla firma a quattro argomenti.
11. Creazione del server HTTP, aggancio del server WebSocket, avvio dei servizi di background.

Perché il log delle richieste sta prima dei router

Registrato dopo il router di autenticazione, `/api/auth/*` non compariva mai nei log: esattamente gli endpoint che servono a diagnosticare un problema di accesso. Inoltre `path` e `ip` sono catturati all'arrivo della richiesta e non nel gestore `finish`, perché un router montato riscrive `req.url` mentre è in esecuzione.

3.1.1 Arresto controllato

Alla ricezione di `SIGTERM` il processo ferma i servizi di background, chiude il server WebSocket, smette di accettare nuove connessioni HTTP, attende il drenaggio di quelle in corso, chiude il pool e termina. Un temporizzatore di **30 secondi** forza l'uscita se qualcosa non si chiude.

3.2 Configurazione

`config.ts` è l'unico punto che legge `process.env`. Ogni valore ha un default esplicito e un commento sul perché di quel default. I valori che riguardano la sicurezza hanno default **chiusi**.

Gruppo	Variabili principali	Note
Processo	<code>NODE_ENV</code> · <code>PORT</code> · <code>LOG_LEVEL</code>	Porta 3000, coerente con il target ingress della Container App.

Gruppo	Variabili principali	Note
Database	DATABASE_URL · DB_SSL_ENABLED · DB_SSL_REJECT_UNAUTHORIZED · SKIP_MIGRATIONS · SEED_ENABLED	TLS attivo in produzione; la verifica del certificato è a <code>true</code> per default.
Controller	WLC_DEFAULT_* · WLC_HTTP_TIMEOUT_MS · WLC_SSH_TIMEOUT_MS · WLC_SSH_VERIFY_HOST_KEY · WLC_TLS_REJECT_UNAUTHORIZED	I valori <code>DEFAULT</code> servono solo allo sviluppo locale: in esercizio i parametri vengono dal record di sede.
SSO	SAML_ENTRY_POINT · SAML_ISSUER · SAML_CALLBACK_URL · SAML_CERT · SAML_LOGOUT_* · SAML_DISABLE_REQUESTED_AUTHN_CONTEXT · SAML_WANT_AUTHN_RESPONSE_SIGNED	SSO è considerato configurato se <code>SAML_ENTRY_POINT</code> è valorizzato.
Emergenza	BREAKGLASS_ENABLED · BREAKGLASS_IP_ALLOWLIST · BREAKGLASS_SESSION_TTL_MINUTES · BREAKGLASS_MAX_FAILED_ATTEMPTS · BREAKGLASS_LOCKOUT_MINUTES · BREAKGLASS_MAX_ATTEMPTS_PER_IP · BREAKGLASS_IP_WINDOW_MINUTES · BREAKGLASS_SEED_*	Disattivato per default. Senza <code>BREAKGLASS_SEED_PASSWORD</code> l'account di bootstrap non viene creato: non esiste una password predefinita.
Permessi	RBAC_CACHE_TTL_SECONDS · RBAC_ENFORCEMENT · RBAC_AUTO_ADMIN_PREFIXES · RBAC_AUTO_ADMIN_DOMAINS	Il codice applica i permessi per default; <code>log-only</code> è un'opzione di rollout.
Posta	MAIL_GRAPH_ENABLED · MAIL_GRAPH_TENANT_ID · MAIL_GRAPH_CLIENT_ID · MAIL_GRAPH_CLIENT_SECRET · MAIL_GRAPH_USER_ID · MAIL_GRAPH_FROM_ADDRESS	Senza configurazione Graph il messaggio viene registrato a log (modalità dimostrativa).
Sessione	SESSION_SECRET	Il ripiego vale solo in sviluppo ed è dichiarato come insicuro nel codice.
Directory	DIRECTORY_SEARCH_ENABLED · DIRECTORY_UPN_DOMAINS · DIRECTORY_MAX_RESULTS	Ricerca del referente in Entra ID. Spenta per default: senza il permesso Graph ogni ricerca fallirebbe, e il campo funziona comunque come testo libero. Domini predefiniti <code>dompe.com</code> , <code>ext.dompe.com</code> , al massimo 15 risultati.
Key Vault	KEY_VAULT_URL	Vault dei segreti <code>WLC-PASSWORD-<CODICE></code> . Vuoto significa «solo variabili d'ambiente».
Telemetria	APPLICATIONINSIGHTS_CONNECTION_STRING · ACR_NAME · ACR_IMAGE_TAG · AZURE_REGION	Tutti opzionali.

Tabella 1 — Gruppi di configurazione letti da ``config.ts``.

3.2.1 Risoluzione della password del controller

`wlcPasswordForSede(code)` consulta tre sorgenti, in ordine.

```

1. cache in memoria      <- letta da Key Vault all'avvio, alla ricarica
                           o quando un amministratore l'ha cambiata
2. WLC_PASSWORD_<CODE>   <- variabile d'ambiente (riferimento a segreto ACA)
3. WLC_DEFAULT_PASSWORD  <- ripiego, solo sviluppo locale

Sede  MIL    -> WLC_PASSWORD_MIL · segreto Key Vault WLC-PASSWORD-MIL
```

La funzione non tocca il database: la password **non è mai** letta né scritta in banca dati.

Un riferimento non risolto non è una password

Container Apps non espande la sintassi `@Microsoft.KeyVault(...)`, e un `secretref:` non risolto arriva al container come testo. Entrambi finivano al controller **al posto della password**. La funzione ora li

riconosce e li **ignora**, passando alla sorgente successiva: il collegamento fallisce con un errore di credenziali anziché con un comportamento inspiegabile.

3.3 Accesso al database

`db/index.ts` espone un `DbClient` minimale (`query`, `exec`, `close`) e la fabbrica `createDbPool()`, unico punto che sa come raggiungere il database.

AUTENTICAZIONE

- Se `DATABASE_URL` contiene una password, viene usata direttamente (sviluppo locale).
- Se non la contiene, un **token Entra ID** è ottenuto tramite `DefaultAzureCredential` sullo scope `https://ossrdbms-aad.database.windows.net/.default`.
- Il token non è acquisito una volta sola: `pg` accetta `password` come funzione asincrona e la invoca **per ogni nuova connessione fisica**, così ciascuna si autentica con un token valido.
- Le connessioni hanno una vita massima di **1800 secondi** e un tempo di inattività di **30 secondi**, entrambi ben al di sotto della validità del token (~60 minuti).

Perché non si ricrea periodicamente il pool

La versione precedente sostituiva il pool su un temporizzatore di 45 minuti, ma il `DbClient` ne manteneva il riferimento in una chiusura: dopo il primo rinnovo ogni query falliva con «Cannot use a pool after calling end on the pool», in modo permanente. Il pool viene ora creato una volta e non sostituito mai.

3.4 Autenticazione SAML

`auth/saml.ts` costruisce la strategia passport-saml. Restituisce `null` — senza far cadere il processo — quando il certificato dell'IdP non è utilizzabile: in quel caso le rotte SSO rispondono 501 e l'accesso di emergenza resta disponibile.

Opzione	Valore	Motivazione
<code>wantAssertionsSigned</code>	<code>true</code> (incondizionato)	Le dichiarazioni di identità devono essere firmate, sempre.
<code>wantAuthnResponseSigned</code>	<code>true</code> , sovrascrivibile	Copre l'intero documento. Richiede la Signing Option «Sign SAML response and assertion» su Entra; dove non è modificabile si abbassa con <code>SAML_WANT_AUTHN_RESPONSE_SIGNED=false</code> .
<code>validateInResponseTo</code>	<code>ifPresent</code>	Protezione anti-replay basata sugli identificativi delle <code>AuthnRequest</code> emesse.
<code>audience</code>	<code>SAML_ISSUER</code>	Verifica che l'asserzione sia destinata a questa applicazione.
<code>signatureAlgorithm</code>	<code>sha256</code>	Nessun algoritmo deprecato.
<code>disableRequestedAuthnContext</code>	<code>true</code>	Non vincola il metodo di autenticazione: è una decisione di Conditional Access del tenant.
<code>identifierFormat</code>	<code>persistent</code>	Identificativo stabile del soggetto.
<code>cacheProvider</code>	PostgreSQL, TTL 1800 s	Vedi §3.4.1.

Tabella 2 — Opzioni della strategia SAML e relative motivazioni.

3.4.1 Cache degli identificativi AuthnRequest

`auth/samlRequestCache.ts` implementa il `CacheProvider` di `node-saml` su PostgreSQL (tabella `saml_request_ids`). La cache in memoria predefinita non sopravvive a un riavvio fra il reindirizzamento di accesso e la risposta dell'IdP e non funziona affatto con più repliche: su Container Apps è la normalità. La scadenza è applicata in SQL e il pool è creato in modo differito, per non legare la costruzione della strategia alla disponibilità del database.

3.4.2 Estrazione delle informazioni di identità

Lo UPN è letto in ordine da `.../claims/upn`, da `.../claims/name` quando ha forma di indirizzo (è il default di Entra, mappato su `user.userprincipalname`) e infine dal `NameID`. L'indirizzo di posta resta come ripiego solo quando nessuna dichiarazione porta uno UPN.

3.5 Sessione

`auth/session.ts` costruisce lo store `connect-pg-simple` passandogli un pool costruito da ``createDbPool()``, non una stringa di connessione: il pool interno della libreria non usa TLS e non dispone del token Entra, e contro Azure PostgreSQL non si connette affatto.

Parametro	Valore
Nome del cookie	<code>guestportal.sid</code>
<code>secure</code>	<code>true</code> in produzione
<code>httpOnly</code>	<code>true</code>
<code>sameSite</code>	<code>lax</code>
Durata	24 ore (120 minuti per una sessione di emergenza, configurabile)
<code>resave / saveUninitialized</code>	<code>false / false</code>
Dimensione del pool dello store	5 connessioni dedicate

Tabella 3 — Parametri della sessione.

L'oggetto utente è serializzato **per intero** in sessione: contiene solo attributi di identità, nessun segreto, ed evita una lettura sul database a ogni richiesta. **Il ruolo non è fra questi attributi** (vedi §3.7).

3.6 Accesso di emergenza

`auth/password.ts` implementa l'hashing con **scrypt** delle sole librerie standard di Node — nessuna dipendenza nativa aggiuntiva, che allargherebbe la superficie di supply-chain per un percorso di accesso raro.

Formato memorizzato (auto-descrittivo, i parametri di costo si possono alzare in futuro):

```
scrypt$<N>$<r>$<p>$<salt-base64>$<hash-base64>
```

`N = 16384` `r = 8` `p = 1` `salt = 16 byte` `chiave = 64 byte`
 Memoria richiesta ≈ 16 MiB, sotto il limite `maxmem` di Node (32 MiB).
 Lunghezza minima imposta dalla CLI: 16 caratteri.

Il confronto è a tempo costante. Un valore memorizzato corrotto o troncato produce `false`, mai un'eccezione: un errore 500 sul percorso di accesso rivelerebbe che l'account esiste.

CONTROLLI ATTORNO AL LOGIN DI EMERGENZA

- `BREAKGLASS_ENABLED` — interruttore generale, spento per default.
- `utils/ipAllowlist.ts` — lista CIDR (IPv4/IPv6) valutata **prima di ogni altra cosa**; fuori lista l'endpoint risponde 404, non 403, per non confermarne l'esistenza.
- `utils/loginThrottle.ts` — finestra scorrevole per indirizzo di origine, in memoria e quindi per replica.
- Blocco dell'account in banca dati (`failed_attempts`, `locked_until`): essendo persistito, è condiviso da tutte le repliche.
- Una sola derivazione script viene sempre spesa, anche per un account inesistente, così la latenza non consente l'enumerazione.
- Un unico messaggio d'errore generico per ogni causa di rifiuto; la causa reale finisce nel log di audit.

Ordine dei controlli

Blocco, disabilitazione e scadenza sono valutati **prima** dell'esito della password, così un account già bloccato non incrementa ulteriormente il proprio contatore. Diversamente, un tentativo continuo di indovinare la password estenderebbe il blocco all'infinito e l'operatore legittimo non potrebbe mai attenderne la fine.

3.7 Autorizzazione

Il modello vive in `auth/authorization.ts` (tipi e funzioni pure) e `middleware/authorize.ts` (risoluzione e guardie). È deliberatamente separato dalla sessione.

• Catena di autorizzazione di una richiesta

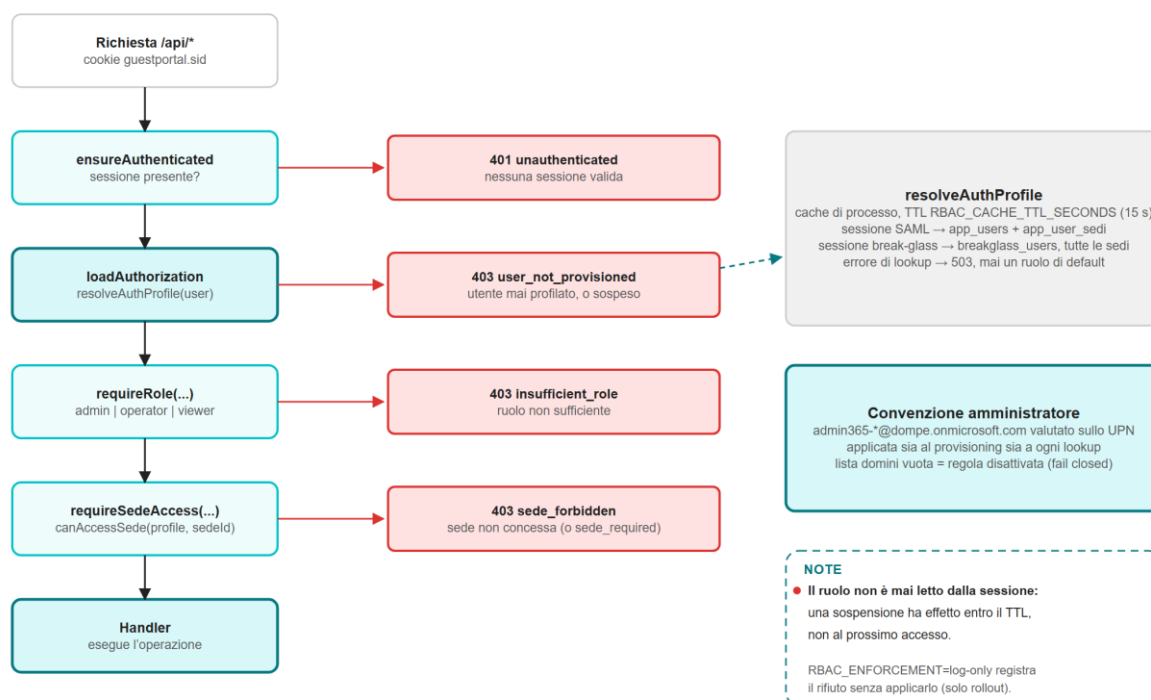


Figura 2 — Catena di autorizzazione: guardie attraversate, rifiuti possibili e sorgenti del profilo.

3.7.1 Il profilo di autorizzazione

```
interface AuthProfile {
  subject: string;           // app_users.subject, oppure bg:<username>
  userId: number | null;     // null per una sessione di emergenza
  source: 'app_user' | 'breakglass';
  role: 'admin' | 'operator' | 'viewer';
  status: 'pending' | 'active' | 'suspended';
  sedeIds: number[];        // sedi concesse (ignorato se allSedi)
  allSedi: boolean;         // amministratori e sessioni di emergenza
  displayName: string;
  email: string;
}
```

`allSedi` esiste perché una sessione di emergenza **non deve dipendere** da righe in `app_users`: è l'accesso che deve funzionare quando il resto è rotto, e una riga malformata nella directory non può essere ciò che chiude l'ultima porta.

3.7.2 Cache e propagazione delle modifiche

Aspetto	Comportamento
Ambito	Cache in memoria, per processo e quindi per replica.
Durata	<code>RBAC_CACHE_TTL_SECONDS</code> , 15 secondi per default — volutamente sotto il polling a 30 s del cruscotto.
Invalidazione esplicita	Ogni modifica fatta da un amministratore invalida la voce sulla replica che l'ha servita.
Assenza di Redis	Scelta deliberata: propagare le invalidazioni fra repliche richiederebbe molte parti mobili per una finestra che il TTL già delimita.
Errore di lookup	503 <code>authz_unavailable</code> . Mai un profilo predefinito.

Tabella 4 — Comportamento della cache di autorizzazione.

3.7.3 Guardie

Middleware	Che cosa verifica	Rifiuto
<code>ensureAuthenticated</code>	Esiste una sessione autenticata.	401 <code>unauthenticated</code>
<code>loadAuthorization</code>	Risolve <code>req.authz</code> ; l'utente è profilato e attivo.	403 <code>user_not_provisioned</code> · 403 <code>user_suspended</code> · 503 <code>authz_unavailable</code>
<code>requireRole(...)</code>	Il ruolo è fra quelli ammessi.	403 <code>insufficient_role</code>
<code>requireSedeAccess(...)</code>	La sede indicata dalla richiesta è fra quelle concesse.	400 <code>sede_required</code> · 403 <code>sede_forbidden</code>
<code>ensureSedeAllowed(...)</code>	Variante per i gestori che scoprono la sede solo dopo una lettura.	403 <code>sede_forbidden</code>

Tabella 5 — Guardie di autorizzazione e codici di rifiuto.

Una sede mancante è un rifiuto, non un valore predefinito

`requireSedeAccess` respinge una richiesta priva di `sedeId`. In passato «nessuna sede» significava «usa la prima riga», ed è esattamente il ripiego che faceva finire l'operazione di un operatore sul controller di un'altra sede.

`allowedSedeIds(profile)` restituisce `null` per «nessuna restrizione» e un array per le sedi concesse. `null` e `[]` significano cose **opposte** e non vanno confusi: il primo è un amministratore che vede tutto, il secondo un utente a cui non è stato concesso nulla e che non deve vedere niente.

3.7.4 Amministratori per convenzione

`isPlatformAdminAddress(upn, { prefixes, domains })` è una funzione pura, condivisa dal percorso di provisioning (che scrive il ruolo nella directory) e dalla risoluzione per richiesta (che lo applica). Due severità deliberate:

- La lista dei domini è **obbligatoria**: vuota, la regola è disattivata anziché aperta a ogni dominio. È il contrario di come si comportano le altre impostazioni a lista, ed è il verso giusto qui — senza il vincolo di dominio un ospite B2B invitato come `admin365-x@attacker.com` arriverebbe come amministratore di piattaforma.
- Dopo il prefisso deve esserci qualcosa: un `admin365-@...` nudo non qualifica.

3.8 Router e gestori

File	Prefisso	Contenuto
<code>routes/auth.ts</code>	<code>/api/auth</code>	Avvio SSO, ACS, logout e SLO, <code>/me</code> , endpoint di emergenza. Montato prima delle guardie generali.
<code>routes/admin.ts</code>	<code>/api/admin</code>	Directory utenti, anagrafica e diagnostica delle sedi, gestione degli account di emergenza. Montato dietro <code>ensureAuthenticated + loadAuthorization + requireRole('admin')</code> ; nulla al suo interno ricontrolla il ruolo.
<code>routes/index.ts</code>	<code>/api</code>	Sonde di salute (aperte), poi tutto il resto dietro autenticazione e autorizzazione: sessione, sedi, WLC, ospiti, configurazioni, storico.

Tabella 6 — Router del backend.

La API amministrativa è tenuta in un file separato **perché l'intera superficie amministrativa possa essere riletta in una sola sessione di revisione** — cosa che una revisione di conformità chiede e che un router da settecento righe non consente.

3.8.1 Persistenza della sessione prima della risposta

`saveSession(req)` attende la scrittura della sessione prima di rispondere. `connect-pg-simple` scrive in modo asincrono e il cruscotto emette la sua prima `GET /guests` nell'istante in cui arriva la risposta: senza l'attesa, quella richiesta può leggere una sessione che non conosce ancora la sede scelta.

3.9 Dialogo con i controller

3.9.1 Verifica delle credenziali via HTTPS

`services/wlcWebui.ts` effettua una `GET` con autenticazione Basic verso la pagina della interfaccia web del controller e ne interpreta la risposta. Un flag `responded` impedisce la doppia risposta nella corsa fra l'evento `error` del socket, la fine della risposta e il timeout.

La classificazione dell'errore è esplicita e non generica: un fallimento TLS **non** viene chiamato «host irraggiungibile».

Categoria	Codici	Messaggio all'operatore
Certificato rifiutato	<code>SELF_SIGNED_CERT_IN_CHAIN</code> , <code>DEPTH_ZERO_SELF_SIGNED_CERT</code> , <code>UNABLE_TO_VERIFY_LEAF_SIGNATURE</code> , <code>CERT_UNTRUSTED</code> , <code>ERR_TLS_CERT_ALTNAME_INVALID</code>	Il controller risponde: a fallire è la verifica del certificato. Nomina la variabile da cambiare.

Categoria	Codici	Messaggio all'operatore
Certificato scaduto	CERT_HAS_EXPIRED	Rinnovare il certificato del controller.
Host irraggiungibile	ETIMEDOUT, EHOSTUNREACH, ENETUNREACH	Verificare egress e routing verso la rete di gestione.
Connessione rifiutata	ECONNREFUSED	L'host risponde ma la porta è chiusa.
Nome non risolto	ENOTFOUND, EAI_AGAIN	Verificare il valore di host.

Tabella 7 — Classificazione degli errori di connessione al controller.

Perché la classificazione conta

Con il certificato autofirmato del Catalyst rifiutato, l'etichetta generica «host irraggiungibile» ha mandato a indagare egress, routing e rete del container mentre il controller rispondeva regolarmente. La classificazione usa `err.code` con ripiego sul testo del messaggio, perché alcune librerie incapsulano l'errore del socket e portano il codice solo nel testo.

3.9.2 Esecuzione dei comandi via SSH

`services/wlcSsh.ts` apre una shell interattiva sul controller IOS-XE ed esegue una sequenza di comandi, riconoscendo il prompt con `/[#>]$/`. Restituisce `{ success, output, error, errorPattern }` e considera fallimento la presenza di modelli di errore IOS-XE nell'output (`% Invalid input detected, % Access denied, % Unauthorized, ...`).

SEQUENZA PER LA CREAZIONE DI UN UTENTE OSPITE

```
terminal length 0
configure terminal
user-name <username>
password 0 <password>
type network-user description Guest-User guest-user lifetime <durata>
description Guest-User
do write memory
end
show running-config | include user-name <username>
exit
```

`minutesToLifetime(minuti)` converte la durata nel formato strutturato atteso da IOS-XE: `year Y month M day D hour H minute M second 0`.

VERIFICA DELLA CHIAVE HOST

- `WLC_SSH_VERIFY_HOST_KEY` è **disattivata per default** (deviazione D2): i cinque controller hanno cinque chiavi diverse a fronte di un unico valore atteso configurabile.
- Quando è attiva **ma** `WLC_SSH_HOST_KEY` manca, la connessione viene rifiutata: connettersi comunque vanificherebbe il senso della richiesta.
- Quando è disattiva, l'applicazione registra una segnalazione **una sola volta** — la sincronizzazione ogni 30 secondi per sede riempirebbe altrimenti i log.

3.9.3 Sanitizzazione

La shell SSH interpreta i comandi riga per riga: un valore che contenesse un ritorno a capo inietterebbe comandi aggiuntivi nella sessione di configurazione. `utils/sanitize.ts` chiude la superficie.

Funzione	Regola
<code>sanitizeIOSXE(v, label, max)</code>	Non vuoto, lunghezza massima, nessun <code>\r \n</code> , nessun byte nullo.
<code>validateUsername(v)</code>	Solo <code>[a-zA-Z0-9._@-]+</code> — copre sia <code>admin_guest</code> sia gli username in forma di indirizzo.
<code>validatePassword(v)</code>	Solo ASCII stampabile (<code>\x20-\x7E</code>): sono bloccati capoversi e caratteri di controllo.
<code>validateHost(v)</code>	Indirizzo o nome host plausibile, usato nei comandi.
<code>validateDurationMinutes(v, min, max)</code>	Intero entro i limiti; il server impone da 1 minuto a 7 giorni .
<code>isLocalUrl(url)</code>	Solo percorsi della stessa origine: impedisce il reindirizzamento aperto sul parametro <code>redirect</code> di <code>/api/auth/login</code> .

Tabella 8 — Funzioni di validazione e sanitizzazione.

Il limite di durata vive sul server

Il modulo di registrazione offre valori predefiniti fino a una settimana, ma il limite è imposto anche lato server: il modulo espose un campo libero in minuti che lo aggirava del tutto, e in ogni caso qualunque client può inviare ciò che vuole.

3.10 Generazione delle credenziali

```
username = "g." + <slug del nome, max 8 lettere> + <3 cifre>    es. g.mrossi417
password = 12 caratteri casuali                                es. Kt7mRq4Wnx2F
```

Alfabeto: ABCDEFGHJKLMNPQRSTUVWXYZ abcdefghjkmnpqrstuvwxyz 23456789
(senza I O i l o 0 1 — si dettano e si trascrivono a voce)
Garanzia: almeno una maiuscola, una minuscola e una cifra, poi mescolate
Sorgente: `crypto.randomInt()` — generatore crittografico, non `Math.random()`

Il prefisso è stato tolto

La password era **DOMPE-** seguito da 8 caratteri casuali: **sei caratteri che qualunque ospite poteva prevedere**, e che la facevano sembrare più lunga di quanto fosse. Ora sono 12 caratteri interamente casuali su un alfabeto di 55 — circa 69 bit — ancora digitabili da un telefono. Gli account già creati mantengono la password che hanno fino al reinvio.

Il carattere di ciascuna classe è inserito per primo e poi mescolato, così la garanzia è rispettata anche con un controller che pretende un insieme misto, senza che le posizioni siano prevedibili.

L'username generato è confrontato con **tutti gli ospiti mai registrati** (`guestUsernameExists`), non solo con quelli della sede: un username riusato erediterebbe sul controller un account creato in precedenza, con la sua durata e la sua scadenza.

3.11 Invio delle credenziali

`services/email.ts` orchestra, `services/graphMail.ts` trasporta. Il trasporto è **solo** Microsoft Graph con il flusso client credentials: SMTP e nodemailer sono stati rimossi, e con essi la tabella di configurazione della posta.

- Se Graph è configurato, l'esito è `{ ok, messageId, mode: 'graph' }` oppure `{ ok: false, error }`.
- Se Graph non è configurato (sviluppo locale), il messaggio viene scritto nei log in modalità `demo-log` e l'esito è positivo.

- Il corpo esiste in versione HTML e testo; tutti i valori dinamici passano da un escape HTML.
- L'esito dell'invio è registrato in `sync_logs` con metodo `GRAPH`.

3.12 Ricerca nella directory Entra ID

`services/entraDirectory.ts` serve il campo **Referente** del modulo di registrazione. Ogni chiamata interroga Microsoft Graph: **nulla è memorizzato**, né sul server né nel browser, così una persona entrata o uscita dal tenant compare o sparisce alla battitura successiva.

```
GET /users
ConsistencyLevel: eventual          (endswith e una query avanzata)
$count = true
$filter = (startswith su displayName, givenName, surname, mail, UPN)
        and (endswith UPN su uno dei domini ammessi)
        and accountEnabled eq true
$select = id, displayName, userPrincipalName
$orderby = displayName $top = DIRECTORY_MAX_RESULTS
```

- L'autenticazione usa l'identità gestita del backend (`utils/azureCredential.ts`), che richiede il permesso applicativo Graph `User.Read.All` con consenso amministratore.
- Il testo cercato è **quotato** per l'OData: gli apici singoli vengono raddoppiati. Lunghezza (2–64 caratteri) e caratteri di controllo sono validati dalla rotta, non dal servizio.
- Il dominio è verificato **due volte**: nel filtro Graph, dove è efficiente, e di nuovo sui risultati, dove non può sbagliare.
- Dal backend escono **solo id** e `displayName`: meno attributi della directory raggiungono il browser dell'operatore, meno ce n'è da perdere.
- Se la ricerca è spenta il servizio solleva `DirectoryDisabledError` e la rotta risponde 503; il campo torna a comportarsi come testo libero.

La query non finisce nei log

È il nome di una persona. In caso di errore viene registrata soltanto la **lunghezza** del testo cercato, insieme all'identificativo di correlazione.

3.13 Password dei controller in Key Vault

`services/wlcCredentials.ts` legge, scrive e ricarica i segreti `WLC-PASSWORD-<CODICE>`; `utils/wlcPasswordCache.ts` è la copia in memoria che `wlcPasswordForSede` consulta per prima.

Funzione	Che cosa fa
<code>readWlcPasswordFromVault(code)</code>	Legge il segreto e aggiorna la cache. Un segreto inesistente restituisce <code>null</code> e rimuove la voce dalla cache.
<code>writeWlcPasswordToVault(code, pwd)</code>	Crea una nuova versione del segreto e aggiorna la cache: il valore è in uso dalla chiamata successiva al controller, senza riavvio.
<code>reloadWlcPasswords()</code>	Rilegge tutte le sedi in parallelo e restituisce <code>{ loaded, missing, failed }</code> . Una sede che fallisce non azzerà il valore che aveva: un errore transitorio non deve mettere fuori servizio un controller funzionante.
<code>initWlcPasswords()</code>	Invocata all'avvio. Non solleva mai: registra l'esito e lascia le variabili d'ambiente come ripego.
<code>describeKeyVaultError(err)</code>	Riassume l'errore dell'SDK in una riga registrabile — stato, codice, prima riga del messaggio, troncata — mai un valore segreto.

Tabella 9 — Funzioni del servizio credenziali.

Perché la cache non importa nulla

`utils/wlcPasswordCache.ts` è deliberatamente privo di import. `config.ts` la legge da `wlcPasswordForSede`, e il servizio che la riempie importa `config.ts`: una dipendenza nel verso opposto sarebbe un ciclo.

La cache è **per processo**. Con una sola replica — la configurazione odierna — è sufficiente; con più repliche una modifica fatta dal pannello raggiunge le altre al riavvio o alla ricarica successiva.

3.14 Servizi di background

• Servizi di background: timer e sincronizzazione con i controller

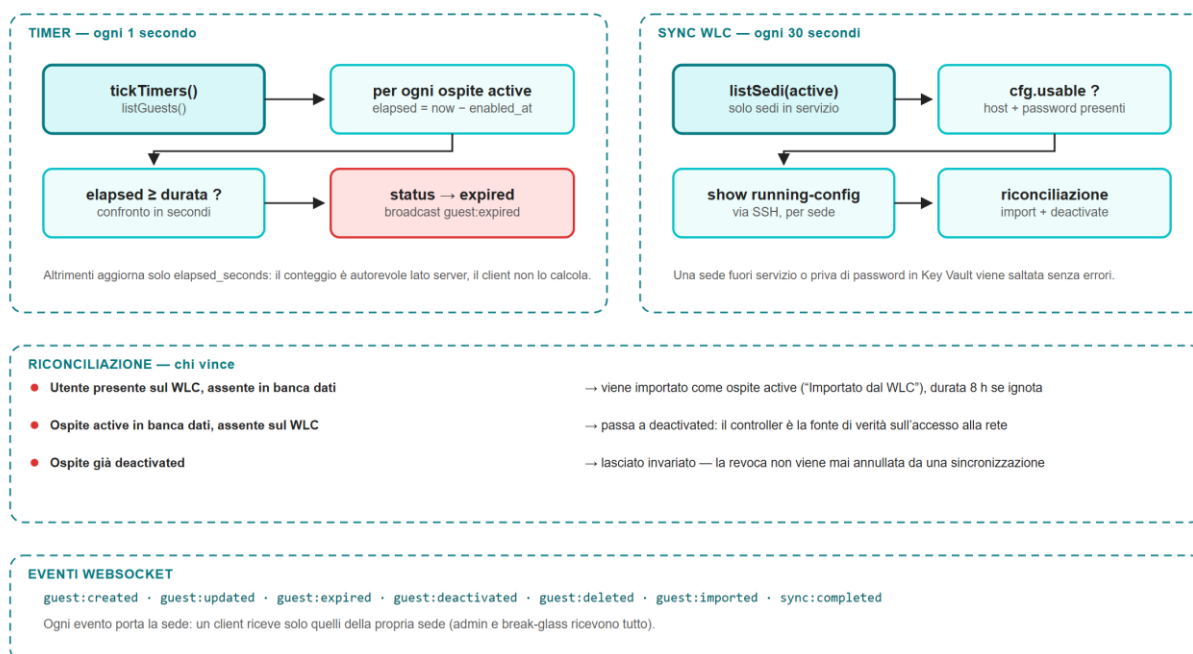


Figura 3 — Timer e sincronizzazione: cadenze, condizioni e regole di riconciliazione.

Servizio	Cadenza	Comportamento
<code>tickTimers()</code>	1 s	Per ogni ospite attivo calcola <code>elapsed = now - enabled_at</code> . Se supera la durata, porta a <code>expired</code> e propaga l'evento; altrimenti aggiorna solo <code>elapsed_seconds</code> .
<code>syncWithAllWlc()</code>	30 s	Per ogni sede in servizio legge la configurazione del controller via SSH e riconcilia: importa ciò che è sul controller e manca in banca dati, disattiva ciò che è in banca dati e non è più sul controller.

Tabella 10 — Servizi di background.

La sincronizzazione salta le sedi per cui `usable` è falso — sede fuori servizio, host non configurato o password assente dall'ambiente. In precedenza il criterio era una colonna `authenticated` che qualunque operatore poteva commutare dall'installazione e che, scrivendo sempre sulla prima riga, poteva fermare in silenzio la sincronizzazione di un'altra sede.

3.15 Provisioning degli account ospite

`services/wlcGuest.ts` raccoglie la sequenza di comandi che prima esisteva in **tre copie** — creazione, reinvio, endpoint amministrativo — e aggiunge la rilettrura che dice se il controller ha davvero ottenuto l'account richiesto.

```
terminal length 0
configure terminal
no user-name <username>          <- solo se replace: true
user-name <username>
type network-user description Guest-User guest-user lifetime <durata>
password 0 <password>
do write memory
end
show running-config | section user-name <username>    <- rilettrura
```

- `type` precede `password`, come nella procedura Cisco: la durata fa parte del tipo, e un account interrotto a metà non deve restare con una password ma senza scadenza.
- La riga `description` ridondante è stata rimossa.
- `replace: true` **elimina e ricrea** l'account. Serve al reinvio: il controller conta la durata da `creation-time`, che una modifica non azzera. Se il controller rifiuta il `no user-name` di un account che non ha, la sequenza viene rieseguita senza.
- La funzione **non solleva mai**: i chiamanti la eseguono senza attenderla, dove un rifiuto non gestito farebbe cadere il processo.

CHE COSA VERIFICA LA RILETTURA

Controllo	Segnalazione
Account presente nella running-config	«account non trovato nella running-config dopo la scrittura»
Blocco <code>type ... guest-user lifetime</code> presente	«account presente ma senza <code>type ... guest-user lifetime</code> »
Durata registrata uguale a quella richiesta	«lifetime registrato N min invece di M»
<code>creation-time</code> presente	«creation-time assente»
<code>creation-time</code> entro 2 minuti dall'orologio del server	«creation-time del WLC avanti/indietro di N min»: account preesistente non ricreato, oppure orologio del controller da verificare
Account non già scaduto	«account già scaduto secondo <code>creation-time + lifetime</code> »

Tabella 11 — Scostamenti rilevati dalla rilettrura.

Gli scostamenti sono registrati a livello `warn` e scritti in `sync_logs` insieme alla **trascrizione della sessione** (ultimi 1.500 caratteri), con ogni password mascherata da `redactTranscript`.

3.16 Presenza dei dispositivi

`services/wlcPresence.ts` risponde a una domanda che prima non si poneva: **qualcuno sta davvero usando questo account?** Lo stato `active` dice solo che l'account è valido.

- La sincronizzazione invia `show wireless client summary detail` come **sonda facoltativa** e ne analizza l'output con `parseWirelessClients`.
- Il risultato è tenuto **in memoria**, per sede, e scade dopo **90 secondi** — tre intervalli di sincronizzazione: oltre quel limite una lettura non dice più nulla sull'adesso.
- Una replica che non ha ancora letto una sede, o la cui lettura è troppo vecchia, risponde **«non noto»**, mai «non connesso».

- Non è mai persistito: è una copia di stato vivo, e salvarla le darebbe solo un modo di diventare obsoleta.

Una sonda che può fallire senza conseguenze

`execSsh` accetta ora `optionalCommands`: un comando `show` che questa versione di IOS-XE non conosce viene segnalato in `optionalFailed` **senza far fallire la sequenza** né buttare via l'output dei comandi riusciti. Il passo di importazione e disattivazione non ne risente. L'assenza della tabella dei client è registrata **una sola volta per sede**, altrimenti la sincronizzazione ogni 30 secondi inonderebbe i log.

3.17 Canale WebSocket

`services/ws.ts` espone `/api/ws`. L'upgrade **non** è automatico: viene intercettato, il cookie di sessione viene letto, la firma verificata con `cookie-signature`, la sessione caricata dallo stesso store usato da Express e l'autorizzazione risolta con `resolveAuthProfile`. Un utente sospeso non conserva il socket.

Ogni connessione porta con sé l'ambito che le è concesso (`sedeId`, `allSedi`) e riceve solo gli eventi di sua competenza: i nomi degli ospiti sono dati personali, e non c'è ragione che un operatore di una sede sappia chi si è appena connesso in un'altra.

```

guest:created      guest:updated      guest:expired
guest:deactivated  guest:deleted      guest:imported
sync:completed    hello
{ type, data: { id, name, username, sedeId }, timestamp }
```

3.18 Registrazione degli eventi

`logger.ts` costruisce un logger **pino** con uscita JSON su stdout, raccolta da Log Analytics. In sviluppo si aggiunge uno stream `pino-pretty`. Se è presente la stringa di connessione di Application Insights e il pacchetto è installato, la telemetria viene inoltrata.

- Ogni riga porta `service`, `env`, `app` e, sul percorso HTTP, `correlationId`.
- La query string è **esclusa** dal log: contiene il parametro `redirect` e la `SAMLRequest`.
- I tentativi di accesso di emergenza, riusciti o meno, sono registrati a livello `warn` proprio perché un alert possa scattare su di essi.
- Nel log dello storico, la password di destinazione è sostituita da `***` prima della scrittura.

3.19 Strumenti da riga di comando

Comando	Scopo
<code>node backend/dist/db/migrate.js</code>	Applica lo schema. Idempotente, esce con 0 o 1. È ciò che esegue il job ACA di migrazione.
<code>node backend/dist/db/seed.js</code>	Popola i dati di riferimento (le cinque sedi). Idempotente per <code>code</code> : non sovrascrive ciò che un amministratore ha corretto.
<code>make breakglass ARGS="..."</code>	<code>list · set · enable · disable · unlock · delete</code> . È l' unico modo per creare un account di emergenza o ruotarne la password.
<code>make appusers ARGS="..."</code>	<code>list · show · grant · suspend · activate · delete</code> . Serve a profilare i primi amministratori su un ambiente nuovo senza esporre l'accesso di emergenza a una rete.

Tabella 12 — Strumenti da riga di comando.

In produzione si eseguono dentro il container, così da ereditare l'identità gestita e il suo token per PostgreSQL:

```
az containerapp exec -n <backend-app> -g <rg> \  
--command "node backend/dist/scripts/appusers.js list --status pending"
```

4 Frontend

4.1 Avvio e macchina a stati

`App.tsx` contiene una macchina a stati esplicita con cinque fasi. Il bootstrap interroga `/api/auth/me` e poi `/api/session/context`, che è la **fonte unica di verità** su utente, permessi e sede corrente.

- Macchina a stati dell'interfaccia e condizioni di accesso

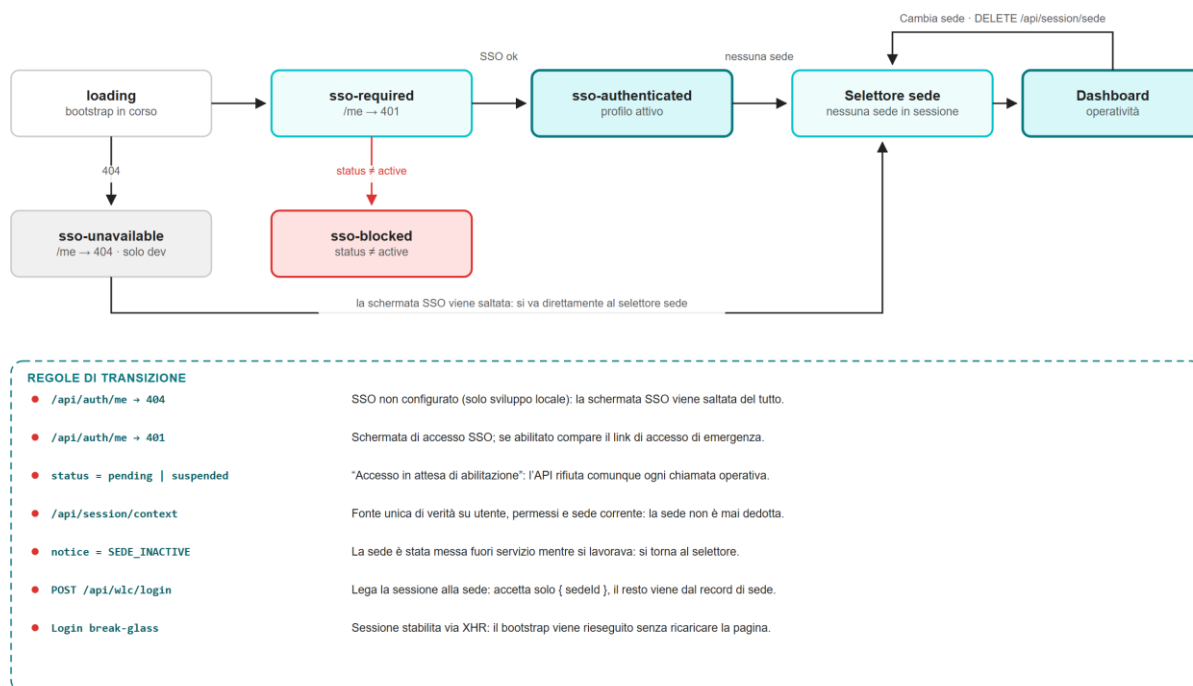


Figura 4 — Fasi dell'interfaccia, transizioni e regole che le governano.

L'interfaccia nasconde, l'API rifiuta

Quando il backend non invia il ruolo — una revisione precedente, o un doppio di test — il client assume «nessuna restrizione» e mostra tutto. Non è una falla: nascondere i comandi è una cortesia verso l'operatore, mentre ciò che impedisce davvero un'operazione è la API.

4.2 Client API

`api/client.ts` espone due oggetti: `api` per la superficie operativa e `adminApi` per quella amministrativa. Gli errori sono incapsulati in `ApiError`, che porta `status` e `code`.

```
class ApiError extends Error {
  status: number; // codice HTTP
  code?: string; // codice applicativo: sede_forbidden, CREDENTIAL_MISSING, ...
}
```

Il codice applicativo esiste perché l'alternativa sarebbe distinguere un rifiuto da un altro confrontando il testo del messaggio. È ciò che consente alla schermata di scelta della sede di dire all'operatore se il problema è di rete, di configurazione o di permessi — tre destinatari diversi.

4.3 Componenti

Componente	Responsabilità
SsoLogin	Schermata di accesso SSO. Mostra il collegamento all'accesso di emergenza solo quando <code>/auth/breakglass/status</code> risponde che è utilizzabile da quel client.
BreakGlassLogin	Modulo di accesso locale; al successo richiede un nuovo bootstrap senza ricaricare la pagina.
PendingApproval	Schermata per l'utente autenticato ma non ancora abilitato; mostra lo UPN, che è ciò che un amministratore riconosce.
Login	Selezione della sede — e nient'altro. In passato conteneva indirizzo, porte, account e SSID del controller, che ogni operatore ridigitava a ogni accesso e che venivano salvati.
Dashboard	Console operativa: elenco ospiti, filtri, stato della connessione, eventi recenti, cambio sede, uscita.
GuestTable	Tabella degli ospiti con durata residua e azioni.
RegisterGuestModal	Registrazione: durate predefinite fino a una settimana o data di fine, e visualizzazione una tantum della password.
DirectoryUserCombobox	Campo Referente con suggerimenti dalla directory: attesa di 300 ms fra una battitura e la richiesta, richiesta precedente annullata (<code>AbortController</code>), da 2 caratteri in su. Navigabile da tastiera. Con la ricerca non disponibile (503) degrada in silenzio a campo di testo .
AdminPanel	Utenti (con eliminazione), sedi — compresa la password del controller: mostra, modifica, ricarica — e account di emergenza.
Toast, UserTag, icons	Presentazione.

Tabella 13 — Componenti del frontend.

4.4 Aggiornamento in tempo reale

`api/ws.ts` apre la connessione a `/api/ws`, espone callback tipizzate e riconnette con attesa esponenziale (fino a 10 tentativi). Il cruscotto reagisce agli eventi ricaricando l'elenco e mantiene un **polling di sicurezza a 30 secondi**: se il canale cade, l'interfaccia resta corretta, solo meno reattiva.

4.5 Internazionalizzazione

`i18n/index.ts` contiene il dizionario italiano e inglese e un hook `useLocale()`. La lingua si cambia senza ricaricare la pagina — utile per le sedi di Tirana e San Mateo, che usano la stessa installazione.

4.6 Runtime statico

L'immagine del frontend è basata su `nginxinc/nginx-unprivileged`, che gira come UID 101 e tiene file temporanei e PID sotto `/tmp`. La configurazione è un file vero — ispezionabile e confrontabile — e prevede:

- `/healthz` e `/readyz` di **solo processo**: non devono interrogare il backend, altrimenti un problema della API farebbe cadere anche il pool del frontend sull'Application Gateway.
- `/api/` che risponde 404: se una richiesta API arriva a nginx, significa che l'instradamento dell'Application Gateway non sta funzionando.
- Ripiego su `index.html` per il routing lato client.

5 Contratti dati

I tipi condivisi vivono in `backend/src/types.ts` e sono rispecchiati in `frontend/src/types.ts`.

```
type GuestStatus = 'pending' | 'active' | 'expired' | 'deactivated';

interface Guest {
  id, name, email, phone, company, host, username;
  password: string | null; // SEMPRE null da banca dati
  durationMinutes, elapsedSeconds;
  status: GuestStatus;
  createdAt, enabledAt, remarks;
  sedeId: number | null;
}

interface Sede {
  id, code, name, city, address, createdAt;
  active: boolean;
  wlcHost: string | null; // null = sede creata, controller non ancora configurato
  wlcPort, wlcSshPort, wlcUsername, wlcSsid;
}

interface DirectoryUser { id: string; displayName: string; }
// solo questi due campi lasciano il backend

interface WlcReloadResult {
  loaded: string[]; // sedi lette da Key Vault
  missing: string[]; // sedi senza segreto nel vault
  failed: { code, error }[]; // sedi non leggibili: valore precedente mantenuto
}

interface AdminSede extends Sede {
  credentialConfigured: boolean; // SE esiste una password, mai il valore
  credentialEnvVar, credentialSecretName;
  wlcLastCheckAt, wlcLastCheckOk, wlcLastCheckError;
  updatedAt, updatedBy;
}

interface WlcConfig {
  host, port, sshPort, username, password, wlanSsid, sedeId;
  authenticated: boolean; // stato della SESSIONE, non del controller
  usable: boolean; // sede attiva + host configurato + password presente
}
```

authenticated e usable non sono la stessa cosa

`authenticated` descrive la sessione di **questo** operatore e viene riempito dalla rotta. `usable` descrive se il server può davvero parlare con quel controller, ed è ciò su cui si basano i servizi di background. In passato `authenticated` era una colonna condivisa da tutti gli operatori e scritta da un pulsante nell'interfaccia: premere «Disconnetti» su una sede poteva fermare il provisioning di un'altra.

6 Errori e codici applicativi

Ogni risposta di errore ha forma { `success: false`, `error: <codice>`, `message: <testo>` }. Il codice è stabile ed è ciò su cui il client si basa; il testo è destinato all'operatore.

Codice	HTTP	Significato
unauthenticated	401	Nessuna sessione valida.
user_not_provisioned	403	Autenticato ma non ancora profilato da un amministratore.
user_suspended	403	Profilo sospeso.
insufficient_role	403	Ruolo non sufficiente; la risposta elenca i ruoli richiesti.
sede_required	400	L'operazione richiede una sede e la richiesta non la indica.
sede_forbidden	403	Sede non fra quelle concesse.
sede_mismatch	403	La sede richiesta non coincide con quella della sessione.
authz_unavailable	503	La risoluzione dei permessi è fallita: la richiesta è rifiutata, non assunta.
NO_SEDE_SELECTED	409	Nessuna sede selezionata in sessione.
SEDE_INACTIVE	409	La sede è fuori servizio.
WLC_NOT_CONFIGURED	400	La sede esiste ma non ha ancora un controller configurato.
CREDENTIAL_MISSING	400	Manca <code>WLC_PASSWORD_<CODICE></code> nell'ambiente.
invalid_payload	400	Corpo della richiesta non valido.
invalid_query	400	Testo di ricerca fuori dai limiti (2–64 caratteri) o con caratteri di controllo.
directory_unavailable	503	Ricerca nella directory non abilitata.
directory_error	502	Microsoft Graph non ha risposto o ha rifiutato la richiesta.
keyvault_unavailable	503	<code>KEY_VAULT_URL</code> non impostato.
keyvault_error	502	Chiamata a Key Vault fallita: per una scrittura, tipicamente il ruolo mancante.
secret_not_found	404	Il segreto della sede non esiste nel vault.
not_found	404	Risorsa inesistente.

Tabella 14 — Codici di errore applicativi.

7 Test

La suite non richiede infrastruttura: i test unitari girano a vuoto, senza database né controller. Un conteggio statico dei casi dichiarati dà **765 test unitari** (570 backend su 32 file, 195 frontend) e **31 casi end-to-end**; va riconfermato con `make test`.

Livello	Strumento	Copertura
Unità e integrazione backend	Vitest + Supertest	32 file: rotte operative e amministrative, autorizzazione, SAML (strategia, callback, cache, UPN), sessione, break-glass (password, blocco, allowlist, throttle), migrazioni, servizi WLC (SSH, chiave host, WebUI), posta, timer, WebSocket, sanitizzazione, credenziali, ricerca nella directory (<code>entraDirectory.test.ts</code>) e password in Key Vault (<code>wlcCredentials.test.ts</code>).
Unità frontend	Vitest + Testing Library	App e macchina a stati, Dashboard, GuestTable, RegisterGuestModal, DirectoryUserCombobox , Login, SsoLogin, BreakGlassLogin, PendingApproval, AdminPanel, Toast, client WebSocket.
End-to-end	Playwright	<code>calendar.spec.ts</code> (durate e data di fine), <code>rbac.spec.ts</code> (visibilità e rifiuti per ruolo), <code>sso.spec.ts</code> (schermata di accesso SSO).

Tabella 15 — Livelli di test.

WORKFLOW DI VERIFICA

- `ci.yml` — controllo dei tipi e test unitari su ogni push.
- `e2e-postgres.yml` → `e2e-reusable.yml` — suite end-to-end completa con PostgreSQL.
- `e2e-sso.yml` — verifica leggera della schermata SSO, solo frontend.
- `docker-security.yml` — scansione Trivy autonoma su ogni push e pull request.

8 Immagini

Aspetto	Backend	Frontend
Base di build	node:22-alpine	node:22-alpine
Base di runtime	node:22-alpine + apk upgrade	nginxinc/nginx-unprivileged:1.28-alpine
Utente	app (non-root, creato nel Dockerfile)	UID 101
Porta	3000	3000
Contenuto copiato	Solo <code>backend/dist</code> + dipendenze di produzione	Solo <code>frontend/dist</code>
Healthcheck	GET <code>/api/healthz</code> via <code>node -e</code> , senza curl	wget su <code>/healthz</code> (non usato da Container Apps, che applica le proprie sonde)
Installazione	<code>npm install --omit=dev</code>	<code>npm ci</code> con lockfile obbligatorio

Tabella 16 — Caratteristiche delle due immagini.

Il lockfile del frontend è copiato senza glob di proposito: la sua assenza deve far **fallire** la build, non produrre in silenzio un albero di dipendenze non riproducibile.

9 Osservabilità

Segnale	Dove	Uso
Log di richiesta	stdout JSON → Log Analytics	<code>correlationId</code> , metodo, percorso, stato, durata, user-agent, IP. Ricostruisce l'intero percorso di una richiesta.
Log di autorizzazione	<code>event: authz-denied / authz-would-deny</code>	Motivo, soggetto, percorso. In modalità <code>log-only</code> registra ciò che sarebbe stato rifiutato.
Log di emergenza	<code>event: breakglass-login-*</code> a livello <code>warn</code>	Base per l'alert obbligatorio su Log Analytics.
Log delle password WLC	<code>event: admin-wlc-password-viewed · -changed · admin-wlc-reload</code> , a <code>warn</code>	Attore, sede, identificativo di correlazione. Una lettura è tracciata come una scrittura. Il valore non compare mai.
Errori Key Vault	<code>event: admin-wlc-keyvault-error</code>	Operazione, sede e messaggio sintetico dell'SDK — che nomina il ruolo mancante, l'informazione che serve a chi deve intervenire.
Storico operativo	Tabella <code>sync_logs</code>	Azione, metodo (SSH/GRAPH), destinazione, codice di stato. Consultabile dalla console.
Sonde	<code>/api/healthz</code> (liveness), <code>/api/readyz</code> (readiness, verifica il database)	Usate da Container Apps e dall'Application Gateway. Restano aperte, senza autenticazione.
Telemetria	Application Insights, opzionale	Attiva solo se la stringa di connessione è presente.

Tabella 17 — Segnali di osservabilità.